FIAP ON

IA COMO FERTILIZANTE DIGITAL - UM NOVO AGRONEGÓCIO DO AMANHÃ

INFRAESTRUTURA VIVA: MICROSSERVIÇOS E DEPLOYS INTELIGENTES COM CLOUDFORMATION

6

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 - Projeto de uma Solução de uma aplicação web conteinerizada 6
Figura 2 - Aplicação de exemplo a ser implantada.....7
Figura 3 - Visão da aplicação de sensoriamento em produção.....7
Figura 4 - Virtual Machine *versus* Contêiner 9
Figura 5 - Imagens em um repositório privado 12
Figura 6 - Versionamento de imagens de contêineres 13
Figura 7 - Tela inicial do Docker Hub 14
Figura 8 - Filtro de categorias no Docker Hub.....15
Figura 9 - Criação de Cluster e Definição de Tarefas 16
Figura 10 - Criação de cluster no ECS.....17
Figura 11 - Cluster em execução no ECS 17
Figura 12 - Docker Hub 18
Figura 13 - Dashboard ECS 19
Figura 14 - Criar nova definição de tarefa 19
Figura 15 - Tarefas e requisitos de infraestrutura.....20
Figura 16 - Especificação de contêiner 21
Figura 17 - Executar nova tarefa.....22
Figura 18 - Configuração de Tarefa 22
Figura 19 - Tarefa em execução 23
Figura 20 - Configuração da Tarefa 23
Figura 21 – Edição de regras de grupo de segurança padrão 24
Figura 22 - Liberação de portas no Grupo de Segurança 24
Figura 23 - Aplicação disponível 25
Figura 24 - Contêineres em execução 25
Figura 25 - Interromper Tarefa 26
Figura 26 - Deploy de Aplicação com uso de IAC 27
Figura 27 - Estrutura da aplicação no VSCode 32
Figura 28 - Criar pilha.....38
Figura 29 - Especificação de arquivo de template.....39
Figura 30 - Nome da pilha no CloudFormation.....39
Figura 31 - Função de execução do IAM 40
Figura 32 - Tarefa criada no ECS.....40
Figura 33 - Execução de tarefa no ECS.....40
Figura 34 - Aplicação implantada.....41
Figura 35 - Exclusão de pilha no CloudFormation.....41
Figura 36 - Tarefas excluídas após exclusão de pilha do CloudFormation 42

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Estrutura de um template do CloudFormation 28
Código-fonte 2 - Código IAC usando arquivo JSON 30
Código-fonte 3 - Código IAC usando arquivo YAML 31
Código-fonte 4 - agriapp.py.....34
Código-fonte 5 - index.html 37
Código-fonte 6 - Código Dockerfile 38

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 INFRAESTRUTURA VIVA: MICROSSERVIÇOS E DEPLOYS INTELIGENTES COM CLOUDFORMATION 5
1.1 Introdução 5
1.2 Projeto Real 5
2 CONCEITOS DE CONTÊINERES 8
2.1 Conceitos básicos de contêineres 8
2.2 Elastic Container Service (ECS) 10
2.3 Elastic Container Registry (ECR) 11
2.4 Docker Hub 14
3 CRIANDO A SOLUÇÃO 16
3.1 Criação do Cluster da solução 16
3.2 Criação de Definição de Tarefas 18
3.3 Liberação de acesso no Firewall (*Security Group*) 24
3.4 Interrupção de tarefas 25
4 INFRAESTRUTURA COMO CÓDIGO 27
4.1 Introdução ao CloudFormation 27
4.2 Estrutura da pilha do CloudFormation 29
4.3 Estrutura da aplicação de exemplo 32
4.4 Criação da pilha do CloudFormation 38
4.5 Exclusão de pilha do CloudFormation 41
CONCLUSÃO 43
REFERÊNCIAS 44
GLOSSÁRIO 45

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 INFRAESTRUTURA VIVA: MICROSSERVIÇOS E DEPLOYS INTELIGENTES COM CLOUDFORMATION

1.1 Introdução

Os microsserviços são uma abordagem de arquitetura de software onde uma aplicação é dividida em pequenos serviços independentes, cada um com uma funcionalidade específica. Esses serviços podem ser desenvolvidos, implantados, escalados e mantidos de forma autônoma, e que se comunicam entre si através de APIs bem definidas, geralmente usando protocolos como HTTP, gRPC ou mensageria.

Com o uso de microsserviços, é possível obter diversas vantagens como:

- **Autonomia:** cada microsserviço é autônomo e pode ser desenvolvido, testado e implantado de forma independente.
- **Descentralização:** cada microsserviço gerencia seus dados, seja da aplicação ou de um banco de dados.
- **Escalabilidade e Resiliência:** é possível escalar serviços específicos conforme a necessidade, sem ter que escalar toda a aplicação. Além disso, se um microsserviço falhar, parte da aplicação continuará funcionando, sendo necessário realizar uma manutenção apenas no serviço com falha.
- **Desenvolvido em diferentes tecnologias:** cada microsserviço pode ser implementado com a tecnologia ou linguagem de programação desejada.
- **Conformidade com práticas ágeis:** cada equipe pode trabalhar em serviços específicos, sem a necessidade de se preocupar com o desenvolvimento de toda a aplicação.

1.2 Projeto Real

A seguir, podemos visualizar o diagrama do projeto de uma solução orientada a microsserviços para receber imagens aéreas tiradas por um drone para realizar um

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

mapeamento agrícola. A aplicação também poderá exibir as imagens armazenadas pela web e executará dentro de um **contêiner** na AWS.

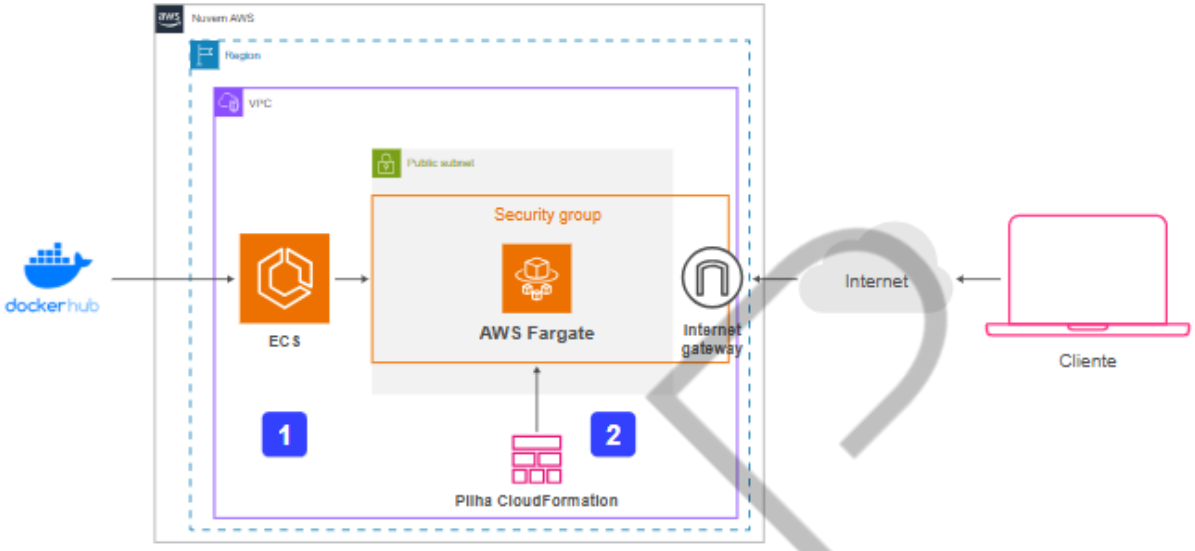

Figura 1 - Projeto de uma Solução de uma aplicação web conteinerizada
Fonte: Elaborado pelo autor (2025), adaptado por FIAP (2025)

Observe a figura anterior e acompanhe o raciocínio: no passo 1 da solução, iremos utilizar uma aplicação web de demonstração para entendermos os conceitos de conteinerização.

Iremos usar o repositório de imagens **Docker Hub** e implantar uma imagem de um contêiner dentro do **Elastic Container Service (ECS)**. Inicialmente, é preciso criar um **cluster** no ECS para conseguir hospedar a aplicação.

Após a conclusão do passo 1, teremos a aplicação funcionando conforme mostra a figura a seguir.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Figura 2 - Aplicação de exemplo a ser implantada
Fonte: Elaborado pelo autor (2025)

No passo 2 da solução apontada na figura “Projeto de uma Solução de uma aplicação web conteinerizada”, iremos implantar uma imagem de uma aplicação web para captura de fotografias tiradas por drones para a tarefa de mapeamento agrícola. A aplicação será executada no serviço **AWS Fargate**, que permite executar contêineres sem a necessidade de gerenciar infraestrutura de servidores diretamente. Iremos implementar a aplicação usando código reprodutível com o uso de uma pilha do **CloudFormation**.

Para o funcionamento da aplicação de drones, configuramos o ***security group*** para permitir acesso público à nossa aplicação, liberando a porta **5000** para a Internet.

A seguir ilustramos como a aplicação aparecerá quando for implantada, após a conclusão do passo 2.

Figura 3 - Visão da aplicação de sensoriamento em produção

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Fonte: Elaborado pelo autor (2025)

Na figura anterior, é possível ver a aplicação em produção que permite escolher arquivos de imagens e incluir a latitude e longitude. As informações são armazenadas em um banco de dados embarcado na própria aplicação, usando o **SQLite**, que permite criar banco de dados em arquivo.

Para implantar esta solução, iremos abordar as características dos serviços envolvidos, e veremos como podemos acionar a automação destes serviços.

Inicialmente, vamos detalhar o conceito de contêineres e quais as diferenças e vantagens em relação ao uso de máquinas virtuais.

2 CONCEITOS DE CONTÊINERES

Considerando a primeira parte de nossa solução, iremos abordar os conceitos básicos de conteinerização e suas principais características.

2.1 Conceitos básicos de contêineres

Os **contêineres** são uma tecnologia de virtualização leve que permite empacotar, distribuir e executar aplicativos de forma isolada, garantindo que eles funcionem de maneira consistente em diferentes ambientes. Ao contrário de máquinas virtuais tradicionais, como no Elastic Compute Cloud (EC2), que virtualiza o hardware, os contêineres compartilham o mesmo sistema operacional, mas isolam os aplicativos em ambientes separados.

O conceito de contêineres surgiu durante a década de 1970 para isolar melhor os códigos de aplicações. No entanto, o conceito se popularizou em 2013 quando o conjunto de produtos de Plataforma como Serviço (PaaS) **Docker** foi lançado, tornando os contêineres mais fáceis de criar, distribuir e gerenciar, se tornando um padrão na indústria para o desenvolvimento de microsserviços e arquitetura *serverless*.

Na figura a seguir, visualizamos as características de contêineres e máquinas virtuais.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

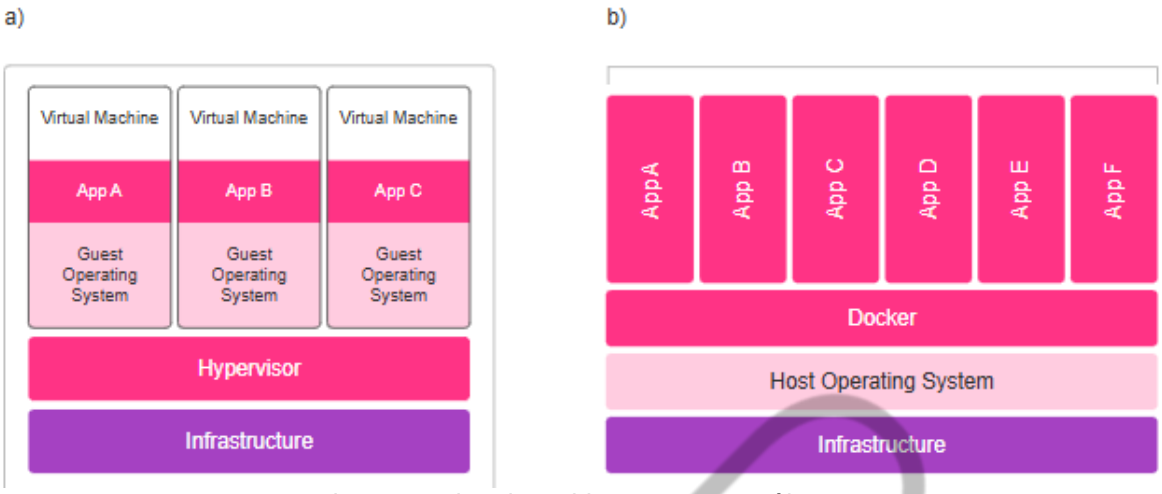

Figura 4 - Virtual Machine *versus* Contêiner
Fonte: Adaptado de Docker.com (2025), adaptado por FIAP (2025)

No lado “a” da figura “Virtual Machine *versus* Contêiner”, podemos observar uma infraestrutura que utiliza máquinas virtuais (Virtual Machine — VM) em que cada máquina virtual contém seu próprio Sistema Operacional (SO), a aplicação necessária e suas dependências, tendo uma camada intermediária que permite rodar mais de uma VM em uma única máquina. Por necessitarem de um SO isolado, as máquinas virtuais ocupam vários Gigabytes (GBs) de espaço em disco e podem ter um processo de inicialização demorado.

Em relação ao lado “b” da figura, é possível observar diversas aplicações, cada uma isolada e que compartilham o kernel do sistema operacional de um hospedeiro. Os contêineres são uma abstração da camada de aplicação que **empacota tanto o código quanto suas dependências**, e que **ocupa menos espaço que as VMs**, podendo iniciar rapidamente como processos isolados na máquina do hospedeiro. Portanto, precisam de menos recursos da máquina hospedeira (host).

Os contêineres facilitam a implantação, pois empacotam todo o código e suas dependências, evitando erros como “funciona na máquina”, mas no servidor de produção, a aplicação não é executada.

Principais características de contêineres:

- **Isolamento:** cada contêiner executa um aplicativo de forma independente, sem interferir em outros contêineres ou no sistema operacional host.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **Portabilidade:** os contêineres podem ser executados em qualquer ambiente que suporte conteinerização (por exemplo: a aplicação Docker), como máquinas locais, servidores de produção ou ambientes de nuvem.
- **Eficiência:** como compartilham o mesmo núcleo do sistema operacional, os **contêineres são mais leves e rápidos do que as máquinas virtuais**, permitindo mais densidade e melhor utilização de recursos.
- **Facilidade de gerenciamento:** as ferramentas de gerenciamento de contêineres, como o Docker, fornecem uma plataforma para construir, testar e implantar aplicativos de forma mais ágil, facilitando a automação de todo o ciclo de desenvolvimento e produção.
- **Imagens e Dockerfile:** os contêineres são criados a partir de imagens, que são especificações estáticas que descrevem o ambiente de execução. Essas imagens são definidas por um arquivo chamado “Dockerfile”, que contém instruções sobre como configurar o contêiner, instalar dependências e executar o aplicativo.

O uso de contêineres confere as seguintes vantagens:

- **Consistência:** evita problemas do tipo funcionar somente na máquina, pois o ambiente de execução é o mesmo em qualquer lugar.
- **Escalabilidade:** é possível replicar contêineres para escalar aplicativos em ambientes distribuídos.
- **Segurança:** o isolamento ajuda a proteger aplicativos de falhas ou ataques que possam ocorrer em outros contêineres ou no sistema host.

A AWS oferece o serviço chamado **Elastic Container Service (ECS)** para hospedar contêineres na nuvem.

2.2 Elastic Container Service (ECS)

O AWS **Elastic Container Service (ECS)** é um serviço gerenciado da AWS para executar, gerenciar e escalar contêineres Docker em clusters de servidores. Ele permite que os usuários implantem aplicativos baseados em contêiner sem precisar

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

configurar infraestrutura subjacente complexa. Isto é, o ECS permite executar contêineres Docker de forma nativa.

Podemos destacar, por exemplo, os seguintes cenários para uso de contêineres na AWS:

- Aplicações web e APIs escaláveis.
- Criação de microsserviços.

Dentro do ECS podemos executar contêineres das seguintes formas:

- **EC2:** cria instâncias EC2 para executar contêineres.
- **Fargate:** serviço gerenciado pela AWS do tipo *serverless*.

Embora o **Fargate** não exija servidores gerenciados pelo usuário (como no modo EC2), ele ainda precisa de um espaço lógico para organizar e rastrear os contêineres em execução. Esse espaço lógico é chamado de **cluster** na AWS. O cluster permite agrupar cargas de trabalho (*workloads*), monitorar aplicações com geração de logs e facilitar a escalabilidade. No entanto, mesmo requerendo uma configuração inicial de um cluster, o gerenciamento do cluster é realizado pela AWS, **sem a necessidade de provisionamento de recursos pelo usuário**.

No AWS Fargate, o cluster é apenas um agrupador lógico, e não um conjunto de servidores. Ele existe para fins organizacionais e de gerenciamento, mas toda a infraestrutura subjacente é gerenciada automaticamente pela AWS.

2.3 Elastic Container Registry (ECR)

O **Amazon Elastic Container Registry (ECR)** é um serviço totalmente gerenciado da AWS para armazenar, gerenciar e distribuir imagens de contêiner. Ele funciona como um repositório de imagens compatível com Docker, permitindo que desenvolvedores e equipes implantem aplicações baseadas em contêineres de forma segura e escalável.

Seus principais recursos são:

- **Suporte a imagens Docker e OCI:** compatibilidade com imagens de contêineres Docker e imagens do Open Container Initiative (OCI).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **Suporte a CI/CD:** permite integração direta com pipelines de implantação com suporte à **integração contínua** (Continuous Integration — CI) e **entrega contínua** (Continuous Development — CD).
- **Busca de imagens sem servidor:** os demais serviços da AWS podem buscar imagens diretamente do ECR sem precisar de servidores adicionais.
- **Armazenamento seguro:** suporte a autenticação integra via IAM e criptografia automática.
- **Versionamento:** mantém múltiplas versões de imagens para fácil recuperação.

O ECR funciona diretamente com o ECS e com funções Lambda.

No ECR, os desenvolvedores podem criar imagens Docker localmente e enviar (*push*) para o ECR em repositórios públicos ou privados.

O ECR é similar ao site Docker Hub que mantém um repositório de imagens públicas e privadas na Internet (https://hub.docker.com/).

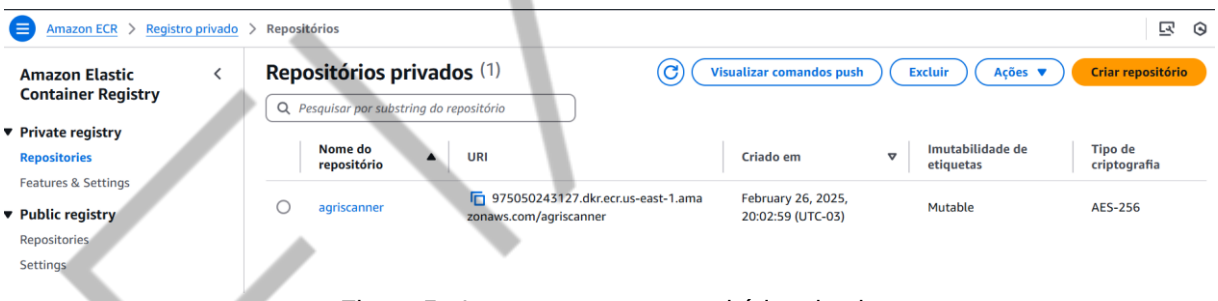

Figura 5 - Imagens em um repositório privado
Fonte: Elaborado pelo autor (2025)

Na figura anterior podemos observar um repositório privado chamado “agriscanner” dentro do ambiente do ECR.

Uma vez que as imagens de contêineres são armazenadas no ECR, é possível versionar as imagens para que possam ser buscadas pelo ECS.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

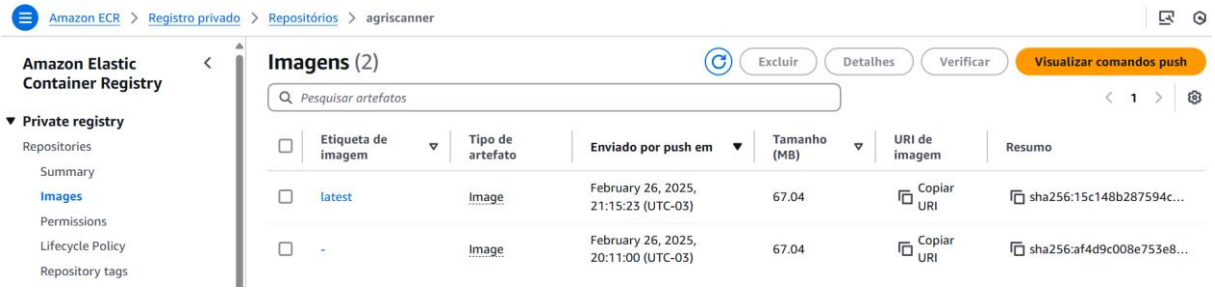

Figura 6 - Versionamento de imagens de contêineres
Fonte: Elaborado pelo autor (2025)

Na figura anterior podemos ver duas versões de imagens de um contêiner, a data de envio da imagem ao ECR e o tamanho da imagem em Megabytes (MB).

Para subir imagens Docker em um repositório ECR, é necessário criar um repositório privado ou público e, em seguida, executar uma série de procedimentos, preparando a imagem para ser enviada ao repositório. Os procedimentos estão listados a seguir:

- **Criar um arquivo Dockerfile:** arquivo de configuração usado para definir a imagem de um contêiner Docker. Contém instruções que determinam como o contêiner será configurado, incluindo sistema operacional, bibliotecas e dependências.
- **Compilar uma imagem Docker:** a partir do comando `docker build` é possível criar uma imagem Docker para poder subir ao repositório ECS.
- **Marcar a imagem com uma tag:** o etiquetamento (tag) deve ser feito com o caminho do repositório no ECR com o comando `docker tag`.
- **Autenticar-se no Amazon ECR:** para enviar a imagem, é necessário realizar a autenticação no repositório ECR.
- **Enviar a imagem para o ECR:** por meio do comando `docker push` é possível enviar a imagem para o ECR.

O ECR apenas armazena imagens, mas não as executa. A execução de contêineres fica a cargo de outro serviço, o Elastic Container Service (ECS).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Portanto, não é possível executar uma imagem diretamente a partir do ECR. Para isso é preciso criar um agrupamento lógico (*cluster*) no ECS para então executar as imagens de contêineres.

Além do ECR, é possível baixar imagens diretamente para o ECS por meio de repositórios públicos na Internet, como o Docker Hub.

2.4 Docker Hub

O Docker é um conjunto de produtos do tipo Plataforma como Serviço (PaaS). O principal registro público de contêineres do Docker é o Docker Hub.

O Docker Hub é um registro de contêineres baseado na nuvem, onde usuários podem armazenar, compartilhar e gerenciar imagens de contêineres Docker. Ele é mantido pela Docker Inc., e é um dos registros de contêiner mais utilizados no mundo.

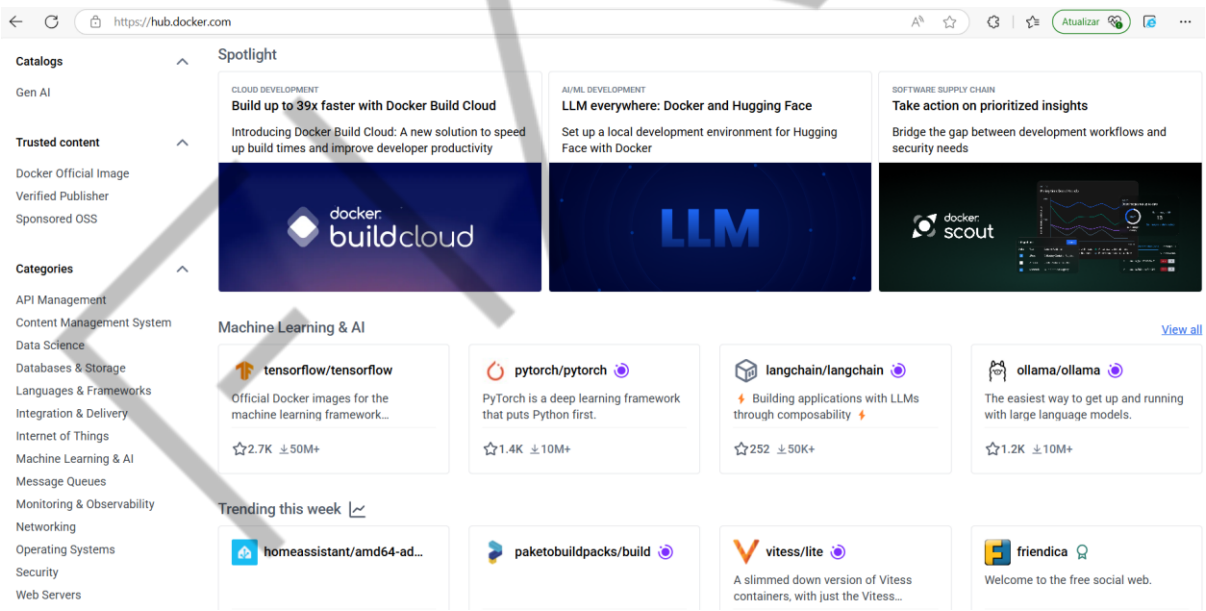

Figura 7 - Tela inicial do Docker Hub
Fonte: Elaborado pelo autor (2025)

Na figura anterior podemos observar a tela inicial do Docker Hub, que você pode acessar através do site: <https://hub.docker.com>.

É possível filtrar por nomes de repositórios de parceiros ou por categorias no menu principal do site.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

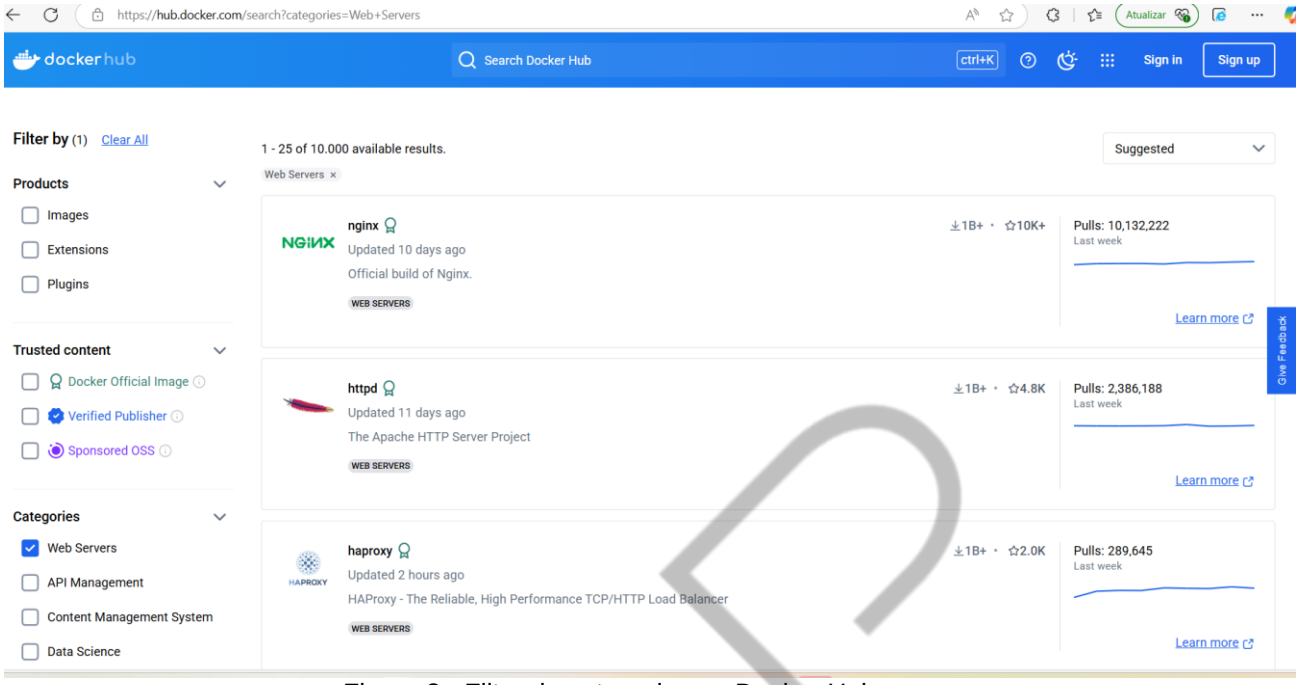

Figura 8 - Filtro de categorias no Docker Hub
Fonte: Elaborado pelo autor (2025)

Na figura “Filtro de categorias no Docker Hub”, selecionamos o menu “Web Servers” para visualizarmos as opções disponíveis de imagens de contêineres, assim como informações de número de downloads (*pulls*) de uma imagem.

Iremos usar imagens oriundas do Docker Hub para baixar no ambiente da AWS e executá-las sem nos preocuparmos em configurar servidores.

A seguir, veremos como construir nossa solução ao qual executaremos uma imagem de contêiner oriunda do Docker Hub.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3 CRIANDO A SOLUÇÃO

Dando continuidade ao nosso projeto, iremos preparar um cluster no ECS para execução de contêineres.

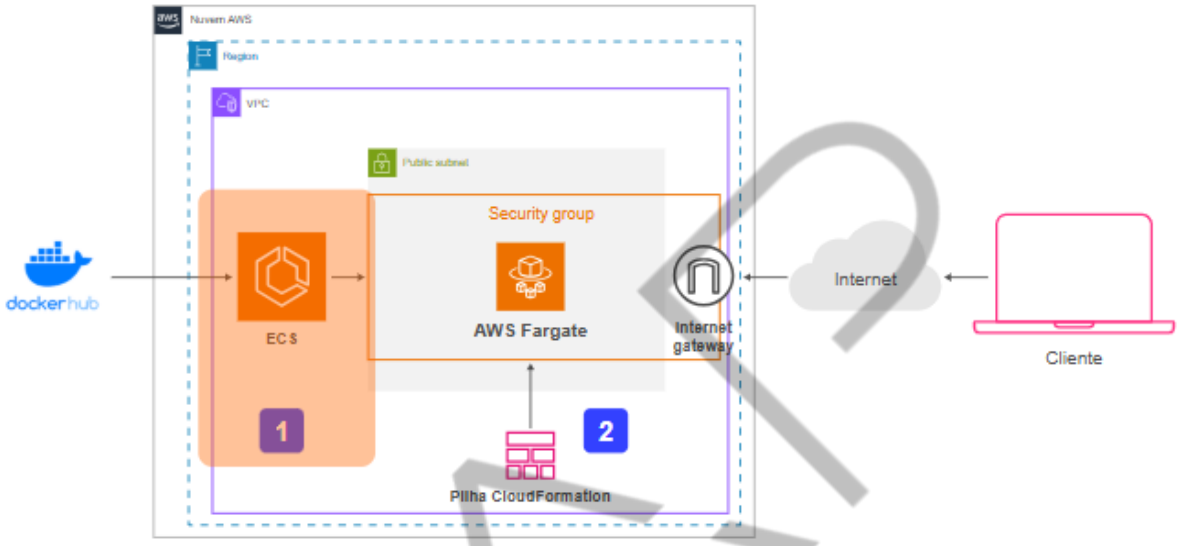

Figura 9 - Criação de Cluster e Definição de Tarefas
Fonte: Elaborado pelo autor (2025), adaptado por FIAP (2025)

Conforme ilustrado na figura “Criação de Cluster e Definição de Tarefas”, iremos preparar um cluster no ECS, que é um agrupamento lógico de contêineres, seguindo o passo 1 da nossa solução.

Com o cluster, será possível executar e gerenciar contêineres oriundos de diferentes repositórios, seja do próprio ECR, ou, no nosso caso, de imagens do Docker Hub.

3.1 Criação do Cluster da solução

Para criar o cluster vamos usar o terminal nativo que está dentro do Console de Gerenciamento, o **CloudShell**. Observe:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

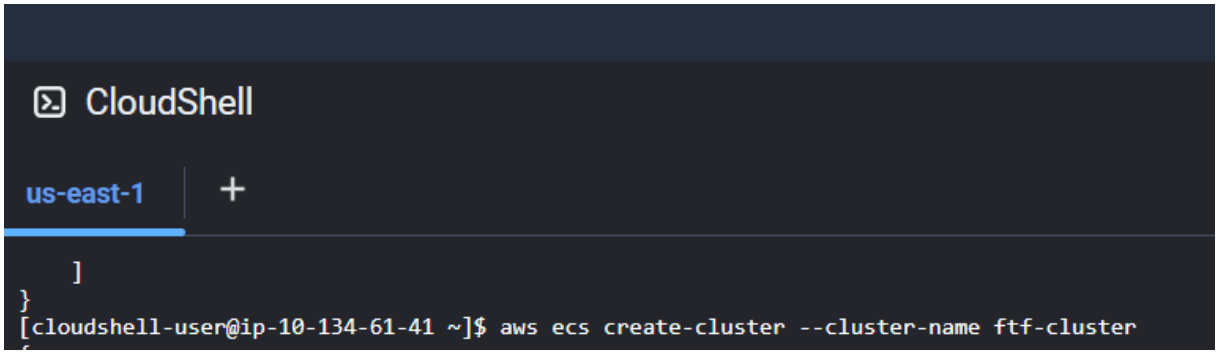

Figura 10 - Criação de cluster no ECS
Fonte: Elaborado pelo autor (2025)

Conforme visto na figura anterior, o comando "`aws ecs create-cluster --cluster-name <nome-cluster>`" permite criar um cluster dentro do ECS.

A partir da criação do cluster, podemos visualizar o cluster em execução e se há tarefas executando.

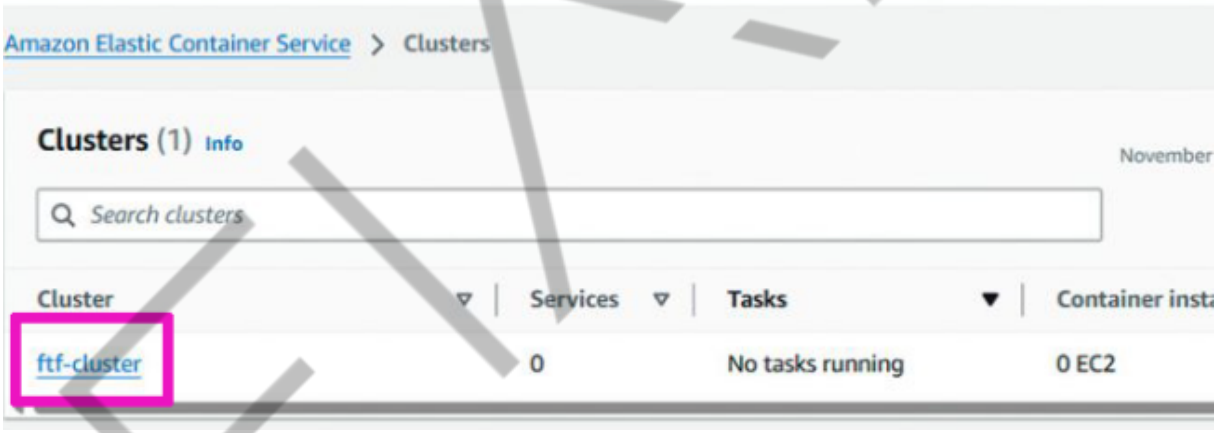

Figura 11 - Cluster em execução no ECS
Fonte: Elaborado pelo autor (2025)

Na figura anterior, é possível observar um cluster em execução sem nenhuma tarefa em execução.

Dentro de um cluster podemos executar diversas tarefas ou serviços. As tarefas são destinadas para execução esporádica de um ou mais contêineres, enquanto um serviço permite uma execução contínua de um determinado número de tarefas como o uso em aplicações web. Um serviço também garante que sempre exista um determinado número de instâncias ativas.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Uma tarefa pode iniciar a execução de um contêiner em uma máquina virtual do EC2 ou de forma *serverless*, dentro do **AWS Fargate** na AWS.

No AWS Fargate não precisamos nos preocupar com gerenciamento de instância EC2, bastando definir quais contêineres que precisamos iniciar.

3.2 Criação de Definição de Tarefas

Iremos criar a nossa primeira definição de tarefa (*task definition*) para realizar a implantação de uma imagem Docker armazenada no repositório de imagens público, **Docker Hub**. Esta ação permitirá entendermos melhor a tecnologia de contêineres utilizada na AWS.

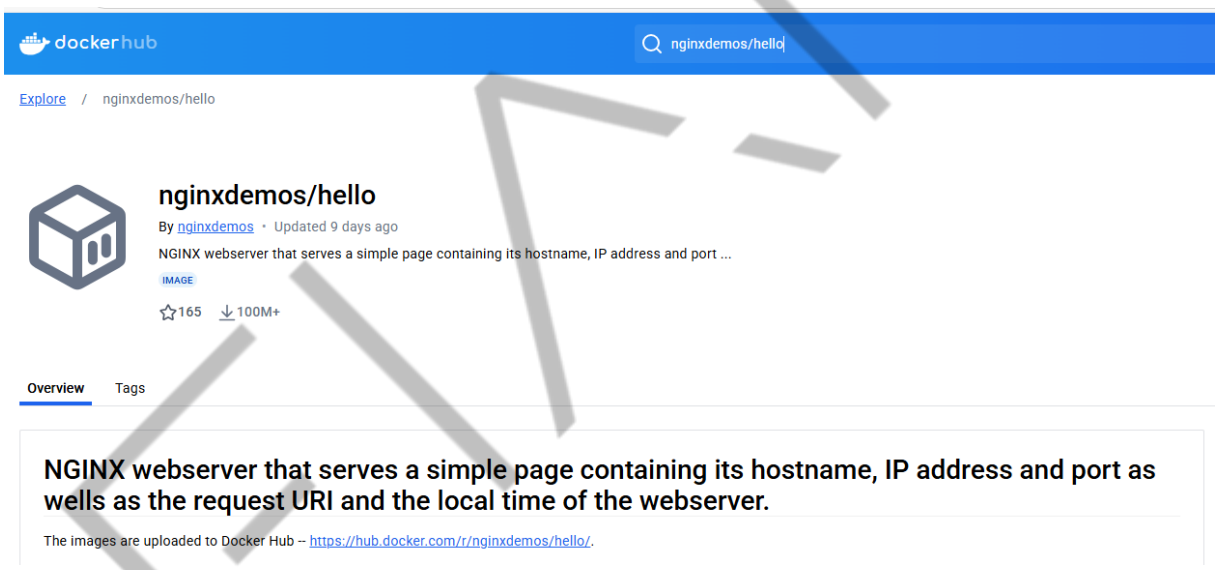

Figura 12 - Docker Hub
Fonte: Elaborado pelo autor (2025)

Na figura anterior, visualizamos o site do Docker Hub onde é possível buscar por diversas imagens de diferentes desenvolvedores.

No exemplo da figura “Docker Hub”, o caminho do “nginxdemos/hello” remete à imagem disponibilizada em repositório público no Docker Hub.

Iremos implantar uma imagem de uma aplicação de exemplo que será hospedada em um servidor web leve chamado **nginx**. Esta imagem será utilizada no AWS Fargate dentro do ECS.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Após o nosso cluster ter sido criado, precisaremos criar uma “**definição de tarefa**”.

Uma definição de tarefa permite criar um modelo (*template*) com informações dos contêineres a serem executados no ECS, além de informar a função de execução (*role*) do IAM que poderá executar tal tarefa.

A definição de tarefas também conta com recursos como versionamento, podendo ser atualizada com novas versões de imagens. A cada nova imagem disponibilizada, são criadas **revisões** (*reviews*) da definição de tarefas.

Digite ECS na caixa de busca no console da AWS e clique em “Definições de tarefa”.

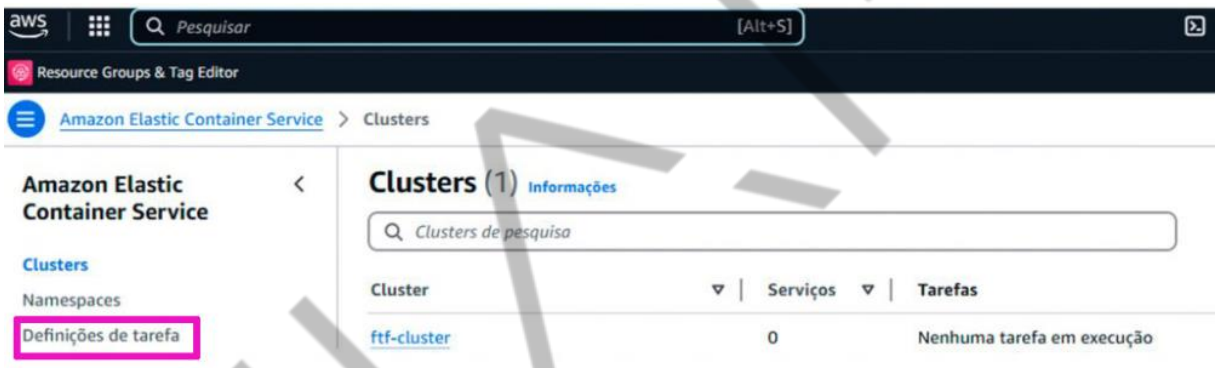

Figura 13 - Dashboard ECS
Fonte: Elaborado pelo autor (2025)

Clique na opção “Definições de tarefa” no menu lateral esquerdo do Elastic Container Service, como ilustrado na figura “Dashboard ECS”. Ao clicar em “Definições de tarefa”, você poderá visualizar todas as definições previamente criadas ou criar novas definições. Em seguida, vá em “Criar nova definição de tarefa”.

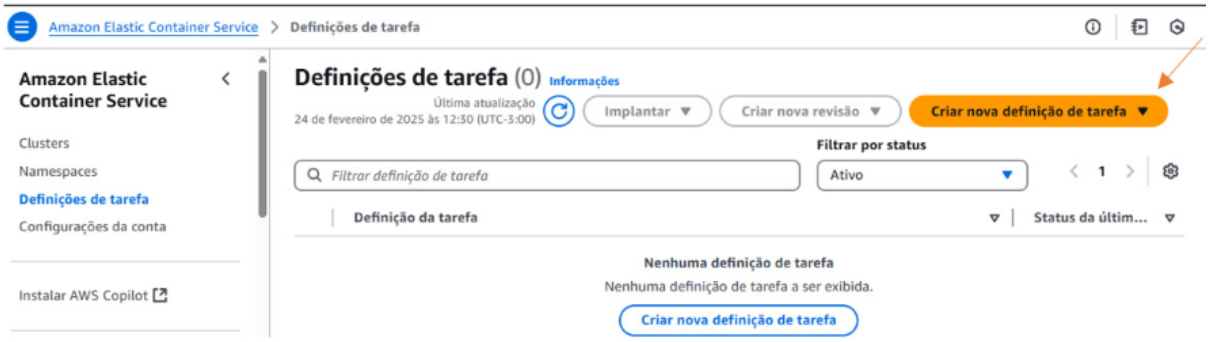

Figura 14 - Criar nova definição de tarefa
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

É possível notar que não temos nenhuma definição de tarefa criada. Então, vamos criar nossa nova definição de tarefa.

Na tela a seguir, clique em “Criar nova definição de tarefa”. Defina o nome da tarefa como “td-helloworld” e mantenha a opção de AWS Fargate. Deixe selecionado o sistema operacional Linux com 1 CPU e 3 GB de memória.

Em seguida, tanto na “Função de tarefa” quanto na “Função de execução de tarefas” selecione a *role* “LabRole”.

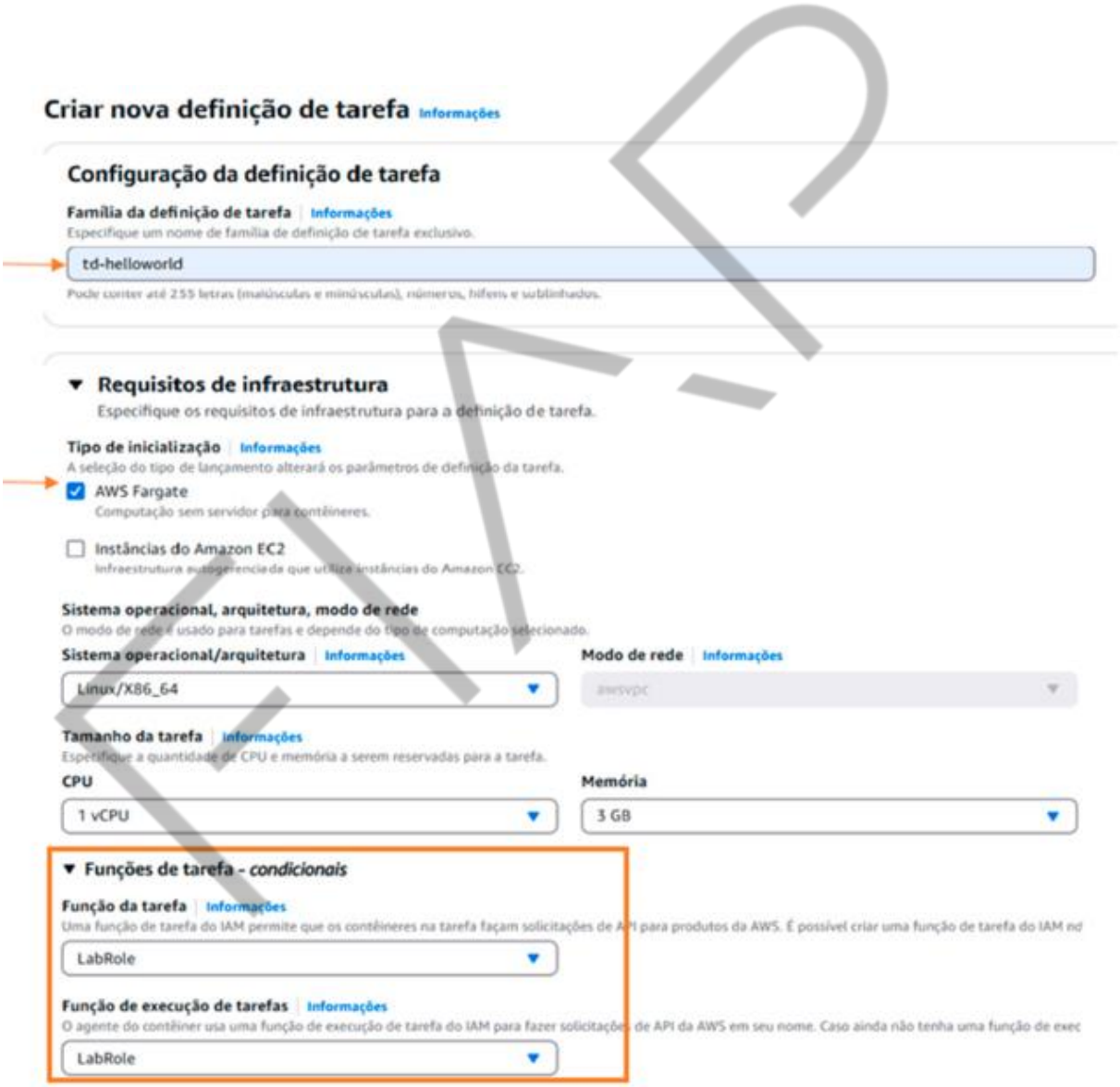

Figura 15 - Tarefas e requisitos de infraestrutura
Fonte: Elaborado pelo autor (2025)

Agora iremos definir os detalhes do contêiner que será implantado. Definiremos um nome, a URI da imagem e as informações, conforme a figura a seguir:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

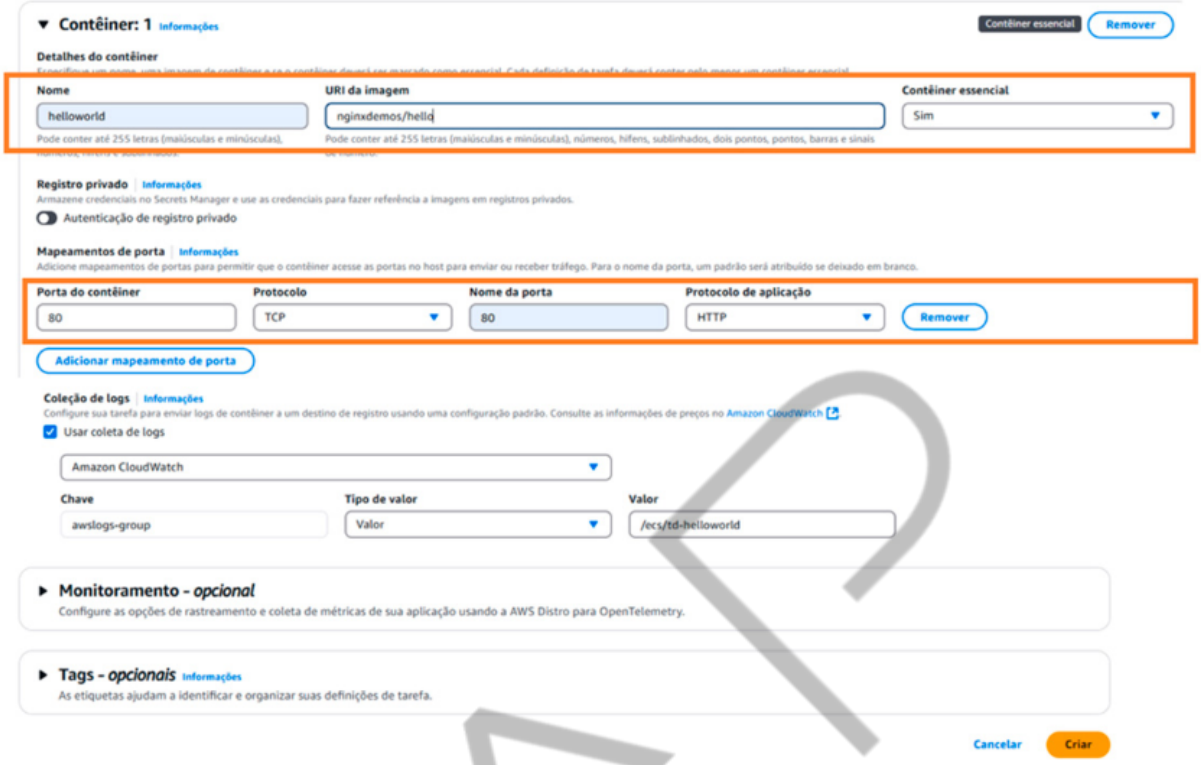

Figura 16 - Especificação de contêiner
Fonte: Elaborado pelo autor (2025)

Na figura “Especificação de contêiner”, omitimos partes que não precisam ser configuradas, tornando necessário preencher apenas os dados do contêiner e clicar no botão “Criar”. No mapeamento de porta, podemos mapear a porta do contêiner para permitir que o contêiner acesse a porta no host. Utilizaremos a porta 80 do Host, por isso definimos a porta do contêiner como 80 e no host também. É possível, inclusive, acessar uma porta diferente no host (exemplo: 8080). Se deixar em branco o nome da porta, um padrão será atribuído.

Agora iremos subir nosso contêiner no cluster previamente criado. Clique no menu lateral em “Clusters”, e selecione o cluster criado. Em seguida, vá em Tarefas > Executar nova tarefa:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Figura 17 - Executar nova tarefa
Fonte: Elaborado pelo autor (2025)

Selecione a “Família” da tarefa e a sua revisão mais recente.

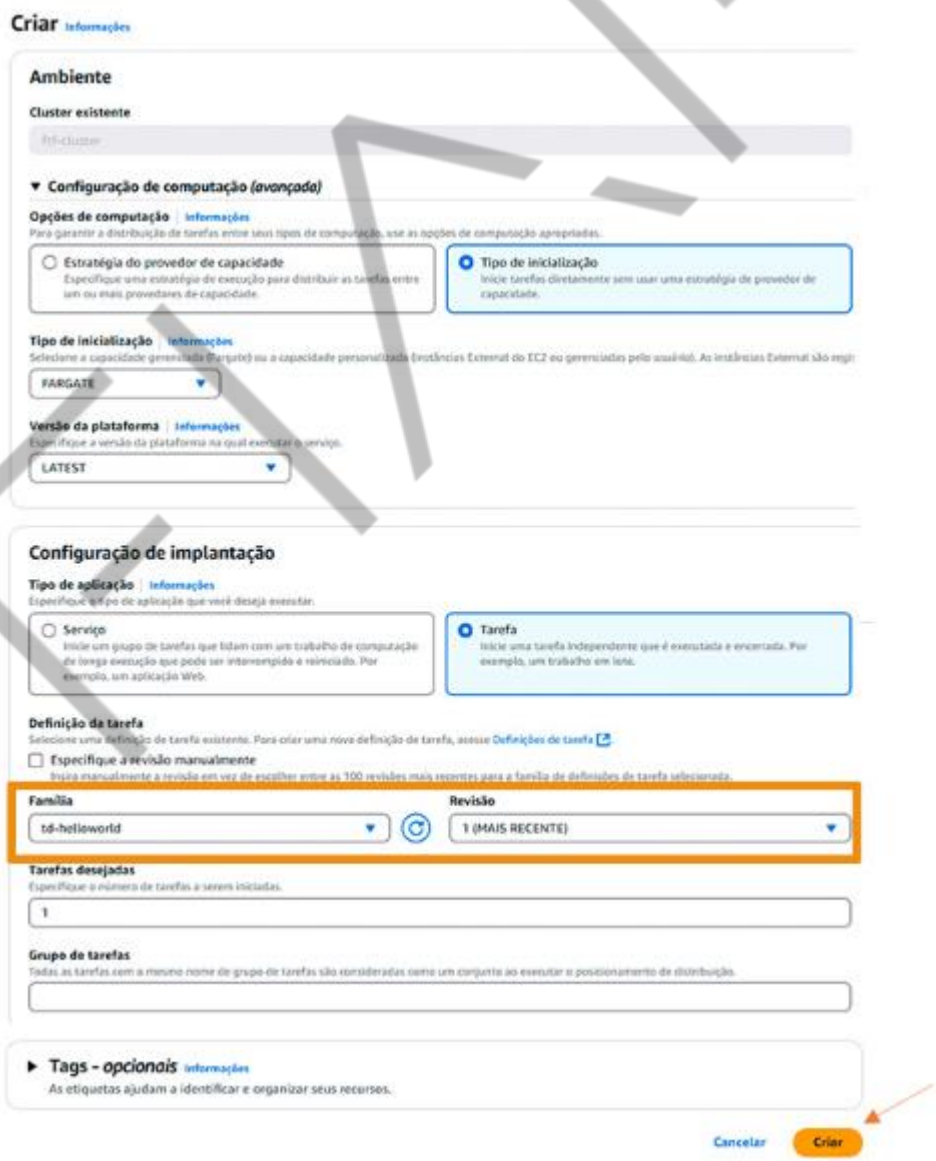

Figura 18 - Configuração de Tarefa
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Aguarde o provisionamento da tarefa. Em seguida, clique no nome da tarefa para resgatar seu endereço público (IP).

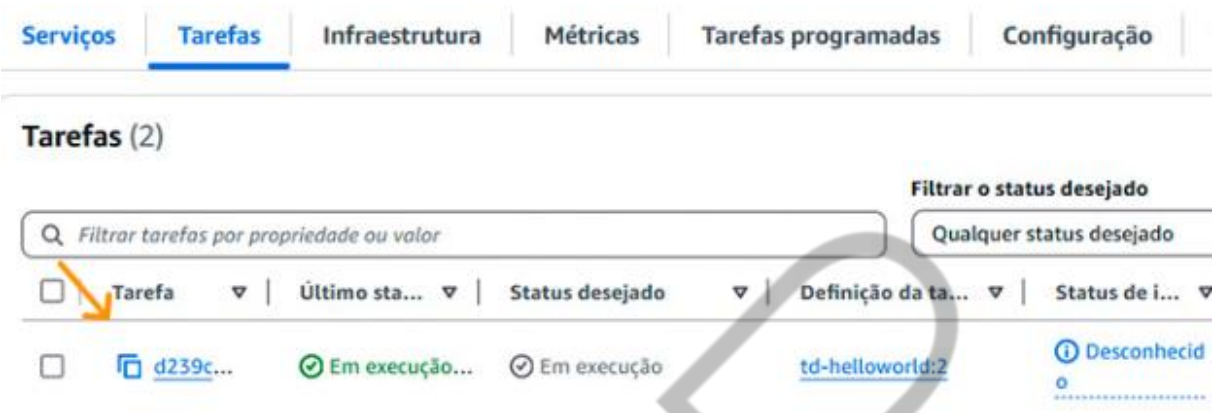

Figura 19 - Tarefa em execução
Fonte: Elaborado pelo autor (2025)

Agora, copie o IP público disponibilizado:

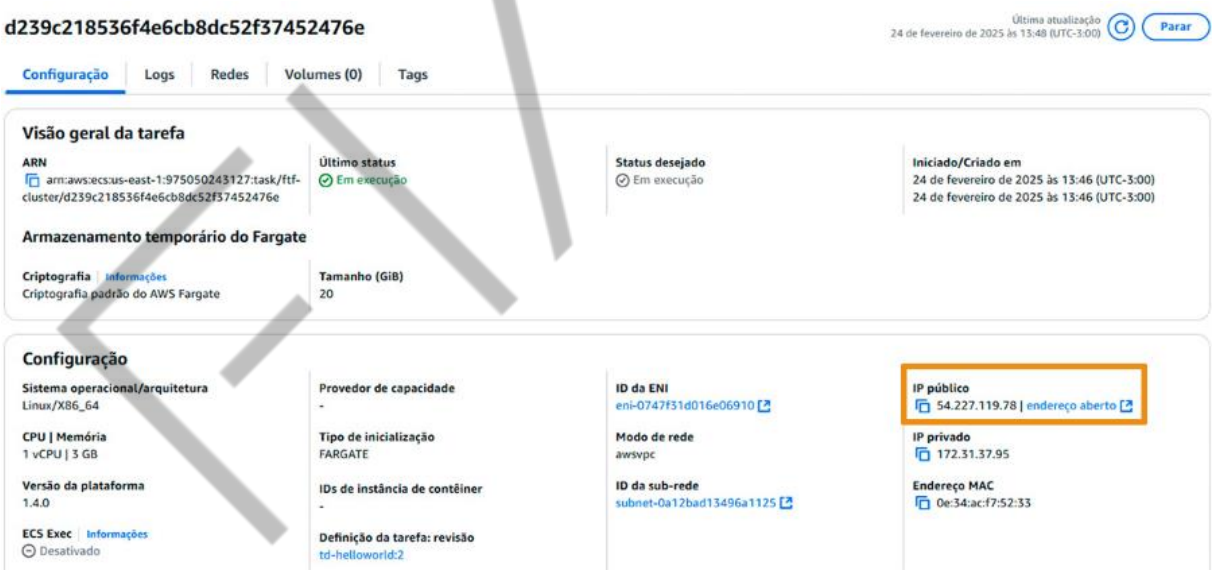

Figura 20 - Configuração da Tarefa
Fonte: Elaborado pelo autor (2025)

Antes de acessar o site <http://IPpúblico>, precisamos liberar o acesso externo para a porta 80, no **grupo de segurança** (*security group*) da nossa VPC padrão.

Vejamos a seguir como liberar a porta 80 no firewall de nossa VPC.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.3 Liberação de acesso no Firewall (*Security Group*)

Nesta etapa, iremos aprender como liberar o acesso em nosso ***security group*** padrão. Para isso, basta buscar por VPC na caixa de busca no console da AWS e procurar por “Grupos de segurança”.

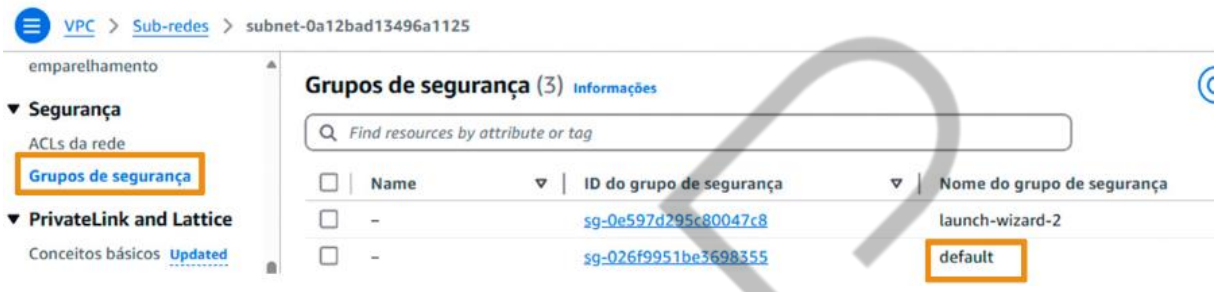

Figura 21 – Edição de regras de grupo de segurança padrão
Fonte: Elaborado pelo autor (2025)

Edite as regras de entrada e libere a porta 80 do grupo de segurança. Para isso, vá até a guia “Regras de entrada” e, em seguida, clique no botão “Editar regras de entrada”.

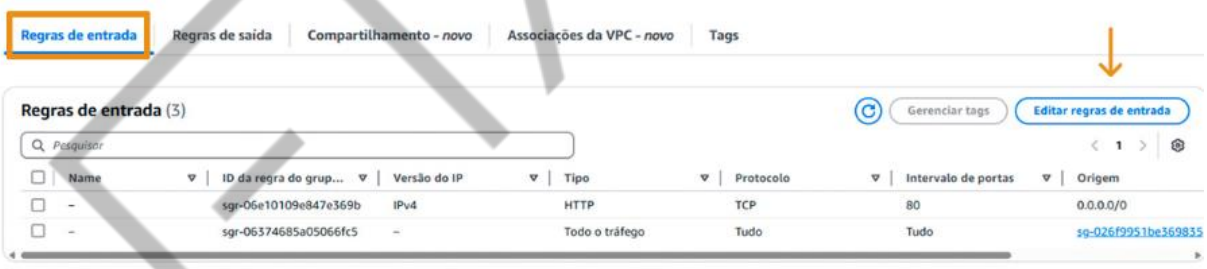

Figura 22 - Liberação de portas no Grupo de Segurança
Fonte: Elaborado pelo autor (2025)

Agora, abra uma nova aba em seu navegador e digite http://IPPúblico do contêiner para visualizar a aplicação funcionando.

Importante: lembre-se de utilizar http e não https no endereço do site.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Figura 23 - Aplicação disponível
Fonte: Elaborado pelo autor (2025)

Concluímos a primeira etapa de nossa solução, em que implantamos uma aplicação demonstrativa para testar os recursos de contêineres na AWS. Agora que finalizamos, iremos interromper a tarefa em execução para encerrar a aplicação.

3.4 Interrupção de tarefas

Após testar e usar a aplicação, o ideal é excluí-la para evitar consumo desnecessário dos recursos da AWS. Ao acessar o ECS, no menu “Clusters”, é possível visualizar as tarefas em execução. Selecione a tarefa e clique em “Parar selecionado(s)”.

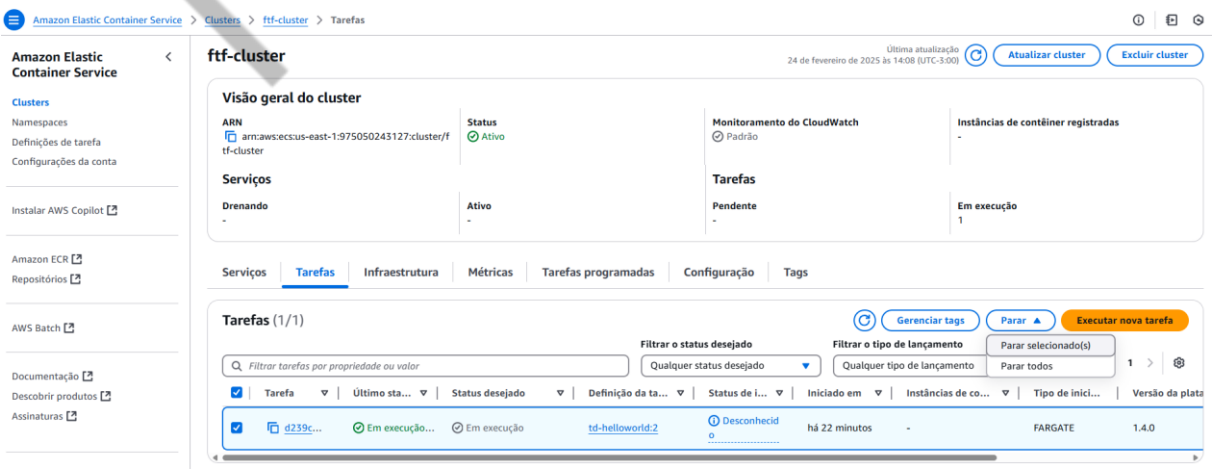

Figura 24 - Contêineres em execução
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Na caixa de diálogo que abrir, confirme o nome da definição de tarefa e clique em “Parar”.

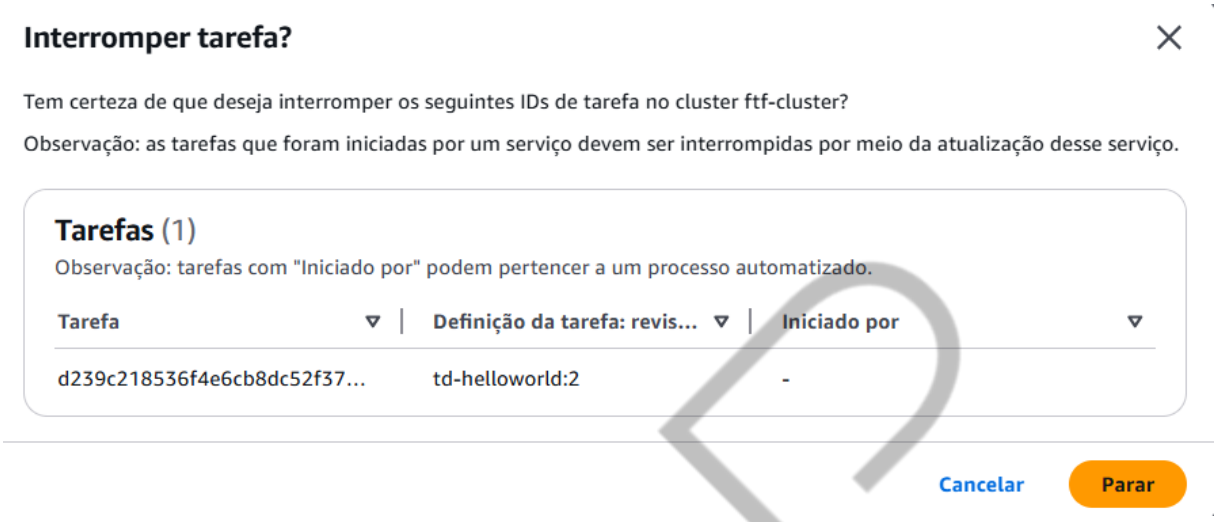

Figura 25 - Interromper Tarefa
Fonte: Elaborado pelo autor (2025)

Pronto! Agora já liberamos os recursos da tarefa. Como estamos utilizando o ECS com o Fargate, não incidirá custos quando não há tarefas em execução.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4 INFRAESTRUTURA COMO CÓDIGO

Vamos implementar a segunda parte de nossa solução. No entanto, iremos subir nossa solução usando infraestrutura como código (Infrastructure as Code — IAC).

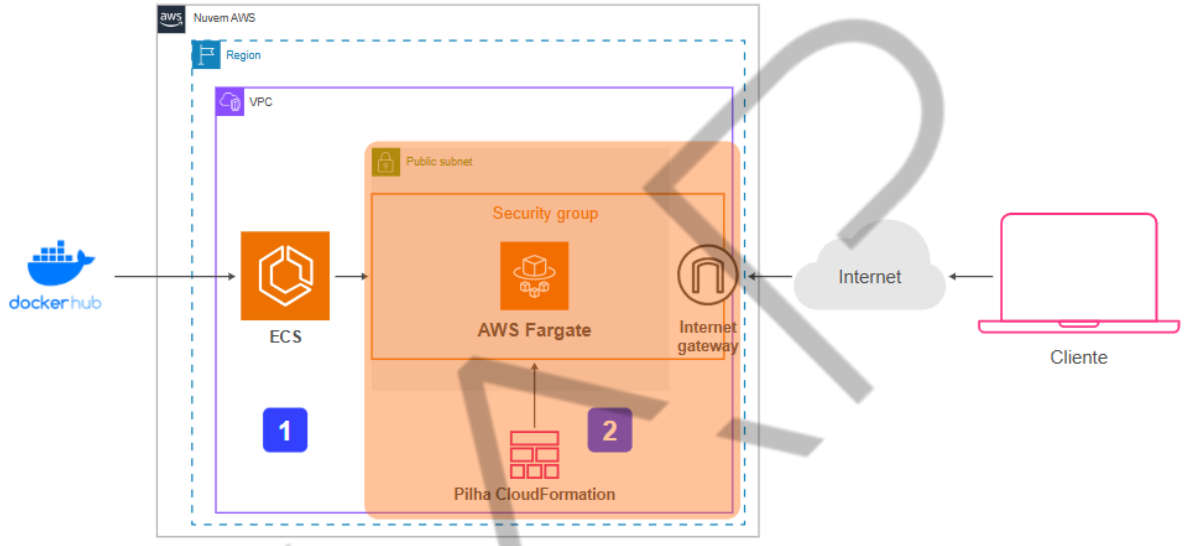

Figura 26 - Deploy de Aplicação com uso de IAC
Fonte: Elaborado pelo autor (2025)

Para simplificar o processo de criação de tarefas e configuração de contêineres, podemos codificar os passos realizados na tarefa em um arquivo reprodutível chamado de *template*. Com o uso de templates, é possível definir etapas como a criação de um cluster, a definição de tarefas e até a configuração de VPCs.

Vejamos como a AWS permite criar templates para implantação de serviços, com o serviço do AWS CloudFormation.

4.1 Introdução ao CloudFormation

O CloudFormation é um serviço que permite configurar a implantação de recursos na AWS a partir de um arquivo de template, que descreve quais e onde os serviços serão implantados. É possível implantar serviços como S3, EC2, RDS, VPC, Elastic Load Balancer (ELB), Lambda, DynamoDB, ECS, além de criação de usuários, *roles*, políticas de acesso entre outros recursos.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

No arquivo de template é definido uma pilha (*stack*) de recursos e suas propriedades a serem implantadas.

```
{
    "AWSTemplateFormatVersion": "data da versão ",
    "Description": "Exemplo de Template",
    "Metadata": { metadados },
    "Rules": { Regras de validação },
    "Parameters": { conjunto de parâmetros},
    "Mappings": {conjunto de mapeamentos },
    "Conditions": { conjunto de condições },
    "Transform": { conjunto de transformações },
    "Resources": { conjunto de recursos },
    "Outputs": { conjunto de saídas }
}
```

Código-fonte 1 - Estrutura de um template do CloudFormation
Fonte: Elaborado pelo autor (2025)

No código anterior, podemos visualizar a estrutura de um template do CloudFormation. É importante ressaltar que existe a obrigatoriedade de definir a **versão** do template e ao menos um **recurso** a ser implantado. Vejamos o que podemos inserir em cada item:

- **AWSTemplateFormatVersion (obrigatório):** identifica a versão do template. A última versão é de 09 de setembro de 2010.
- **Description (opcional):** permite definir um texto de até 1024 bytes.
- **Metadata (opcional):** define informações adicionais sobre o template ou os recursos a serem instalados, como descrição de uma instância ou de uma base de dados.
- **Rules (opcional):** valida valores de parâmetros ou um conjunto de parâmetros durante a criação da pilha.
- **Resources (obrigatório):** especifica a pilha de recursos e as propriedades. Por exemplo: EC2, bucket S3 etc.
- **Parameters (opcional):** permite passar valores ao criar ou atualizar uma pilha. Por exemplo: tipo da instância EC2 ser implantada.
- **Mappings (opcional):** define conjunto de pares de chave-valor para ajustar configurações. Pode-se mapear tipos de instâncias diferentes do EC2 de acordo com a região de implantação da pilha.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **Conditions (opcional):** definem recursos ou propriedades com base em condições. Por exemplo, podemos implantar recursos diferentes em ambientes de produção e desenvolvimento com base no valor de um parâmetro de ambiente.
- **Transform (opcional):** podem definir macros simples, como encontrar e substituir textos.
- **Outputs (opcional):** pode definir saídas como o nome do bucket S3 criado na pilha, para facilitar sua busca.

Depois da implantação da pilha, os recursos estarão disponíveis e executando, o que facilita o gerenciamento, pois evita a necessidade de configurar recursos individualmente.

Além disso, o CloudFormation permite atualizar arquivos de template, para manter um histórico de implantações.

Após o término da utilização dos recursos, para garantir a remoção de todos os recursos usados em uma pilha, basta excluir a pilha do CloudFormation, e todos os recursos vinculados são automaticamente removidos, facilitando a gestão pelas equipes.

4.2 Estrutura da pilha do CloudFormation

É possível utilizar diferentes tipos de formatos para codificar a implantação de recursos. Podemos trabalhar tanto com arquivos JSON ou arquivos YAML. Vejamos a seguir a codificação de um arquivo JSON.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
      {
        "AWSTemplateFormatVersion": "2010-09-09",
        "Resources": {
          "TaskDefinition": {
            "Type": "AWS::ECS::TaskDefinition",
            "Properties": {
              "Family": "td-agriapp",
              "RequiresCompatibilities": [
                "FARGATE"
              ],
              "Cpu": "256",
              "Memory": "512",
              "NetworkMode": "awsvpc",
              "ExecutionRoleArn": {
                "Fn::Sub":
"arn:aws:iam::${AWS::AccountId}:role/LabRole"
              },
              "TaskRoleArn": {
                "Fn::Sub":
"arn:aws:iam::${AWS::AccountId}:role/LabRole"
              },
              "ContainerDefinitions": [
                {
                  "Name": "agriapp-container",
                  "Image":
"proffernandotfernandes/agriscanner",
                  "Essential": true,
                  "PortMappings": [
                    {
                      "ContainerPort": 80,
                      "Protocol": "tcp"
                    },
                    {
                      "ContainerPort": 5000,
                      "Protocol": "tcp"
                    }
                  ]
                }
              ]
            }
          }
        }
      }
```

Código-fonte 2 - Código IAC usando arquivo JSON
Fonte: Elaborado pelo autor (2025)

Agora vamos ver o mesmo arquivo convertido para YAML.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
      AWSTemplateFormatVersion: '2010-09-09'
      Resources:
        TaskDefinition:
          Type: AWS::ECS::TaskDefinition
          Properties:
            Family: td-agriapp
            RequiresCompatibilities:
              - FARGATE
            Cpu: '256'
            Memory: '512'
            NetworkMode: awsvpc
            ExecutionRoleArn:                                   !Sub
arn:aws:iam::${AWS::AccountId}:role/LabRole
            TaskRoleArn:                                        !Sub
arn:aws:iam::${AWS::AccountId}:role/LabRole
            ContainerDefinitions:
              - Name: agriapp-container
                Image: proffernandotfernandes/agriscanner
                Essential: true
                PortMappings:
                  - ContainerPort: 80
                    Protocol: tcp
                  - ContainerPort: 5000
                    Protocol: tcp
```

Código-fonte 3 - Código IAC usando arquivo YAML
Fonte: Elaborado pelo autor (2025)

Perceba que o código é ligeiramente menor, pois não utiliza chaves para agrupar ações relacionadas. Ao invés disso, é utilizada a indentação, que melhora a legibilidade do código.

Veja que no contêiner, definimos que usaremos a definição de tarefa “td-agriapp” criada anteriormente, parametrizando o “FARGATE” para execução *serverless* da tarefa.

Em seguida, devemos definir que iremos utilizar 1/4 de CPU e 512 MB de memória. A AWS usa uma configuração em unidades de CPU em vez de GHz. Usamos a função de execução “LabRole” para a execução da tarefa.

Por último, definimos um nome para o contêiner que será executado, assim como a imagem do contêiner. Neste template, utilizaremos uma imagem pública disponibilizada no Docker Hub no repositório “proffernandotfernandes/agriscanner”.

Para que seja possível acessar o contêiner, precisamos criar o mapeamento para as portas 80 e 5000 TCP.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4.3 Estrutura da aplicação de exemplo

A aplicação de exemplo foi desenvolvida em Python, usando um framework web leve denominado **flask**. A aplicação tem a seguinte estrutura:

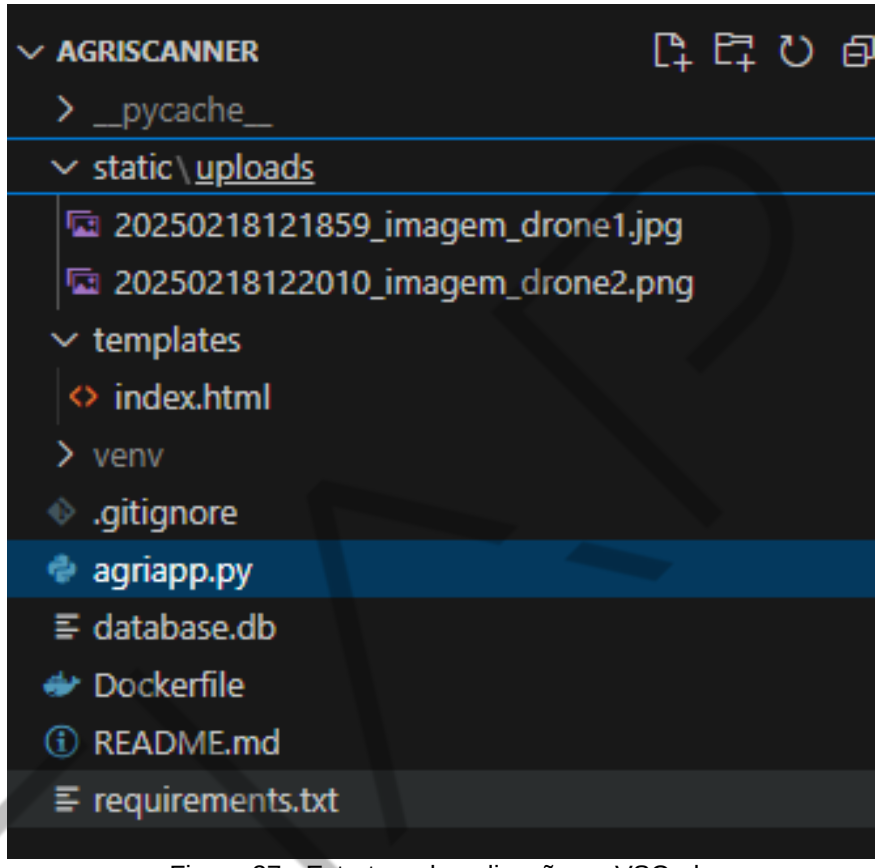

Figura 27 - Estrutura da aplicação no VSCode
Fonte: Elaborado pelo autor (2025)

A aplicação consiste em receber imagens e armazená-las na pasta “static/uploads”.

O código da aplicação fica no arquivo “agriapp.py” e o controle de imagens gravadas é feito por meio do **sqlite** embarcado na aplicação, como no arquivo “database.db”.

A seguir exibimos o código da aplicação.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
     from flask import Flask, request, render_template, redirect,
url_for # type:ignore
     import os
     from datetime import datetime
     import sqlite3

     app = Flask(__name__)
     app.config['UPLOAD_FOLDER'] = 'uploads'
     os.makedirs(app.config['UPLOAD_FOLDER'], exist_ok=True)

     def init_db():
         try:
             # Use an absolute path to ensure it’s in the correct
directory
             db_path = os.path.join(os.getcwd(), 'database.db')

             conn = sqlite3.connect(db_path)
             cursor = conn.cursor()

             # Create table if not exists
             cursor.execute('''
                 CREATE TABLE IF NOT EXISTS images (
                     id INTEGER PRIMARY KEY AUTOINCREMENT,
                     filename TEXT NOT NULL,
                     date TEXT NOT NULL,
                     latitude REAL NOT NULL,
                     longitude REAL NOT NULL
                 )
             ''')
             conn.commit()
             conn.close()
             print("Banco de dados inicializado e tabela 'images'
criada com sucesso!")
         except Exception as e:
             print(f"Erro ao inicializar o banco de dados: {e}")

     @app.route('/')
     def index():
         conn = sqlite3.connect('database.db')
         cursor = conn.cursor()
         cursor.execute('SELECT   filename,   date,   latitude,
longitude FROM images ORDER BY date DESC')
         images = cursor.fetchall()
         conn.close()
         return render_template('index.html', images=images)

     @app.route('/upload', methods=['POST'])
     def upload_file():
         if 'image' not in request.files:
             return redirect(request.url)
         file = request.files['image']
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        latitude = request.form.get('latitude')
        longitude = request.form.get('longitude')
        if file.filename == '' or not latitude or not longitude:
            return redirect(request.url)

        filename  =  datetime.now().strftime('%Y%m%d%H%M%S')  +
'_' + file.filename

        app.config['UPLOAD_FOLDER'] = 'static/uploads'
        os.makedirs(app.config['UPLOAD_FOLDER'],
exist_ok=True)

        file_path  =  os.path.join(app.config['UPLOAD_FOLDER'],
filename)
        file.save(file_path)

        conn = sqlite3.connect('database.db')
        cursor = conn.cursor()
        cursor.execute('INSERT  INTO  images  (filename,  date,
latitude, longitude) VALUES (?, ?, ?, ?)',
                        (filename,  datetime.now().isoformat(),
latitude, longitude))
        conn.commit()
        conn.close()

        return redirect(url_for('index'))

    @app.before_request
    def before_request():
        init_db()

    if __name__ == '__main__':

        app.run(host='0.0.0.0', port=5000)
```

Código-fonte 4 - agriapp.py
Fonte: Elaborado pelo autor (2025)

Conforme visto no código-fonte “agriapp.py”, a aplicação cria rotas para envio de requisições via POST. Além disso, permite a visualização das imagens a partir da renderização em um arquivo “index.html”.

A seguir exibimos o código html.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
    <meta charset="UTF-8">
    <meta   name="viewport"   content="width=device-width,
initial-scale=1.0">
    <title>Galeria de Imagens</title>
    <style>
        * {
            box-sizing: border-box;
            margin: 0;
            padding: 0;
            font-family: Arial, sans-serif;
        }

        body {
            background-color: #f4f4f4;
            padding: 20px;
        }

        h1 {
            text-align: center;
            margin-bottom: 20px;
        }

        form {
            display: flex;
            justify-content: center;
            gap: 10px;
            margin-bottom: 20px;
            flex-wrap: wrap;
            background: #fff;
            padding: 15px;
            border-radius: 10px;
            box-shadow: 0px 0px 10px rgba(0, 0, 0, 0.1);
        }

        input, button {
            padding: 10px;
            border: 1px solid #ccc;
            border-radius: 5px;
            font-size: 14px;
        }

        input {
            width: 200px;
        }

        input[type="file"] {
            width: 100%;
            max-width: 350px;
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
}


button {
    background-color: #28a745;
    color: white;
    border: none;
    cursor: pointer;
}

button:hover {
    background-color: #218838;
}

h2 {
    margin-bottom: 10px;
    text-align: center;
}

.gallery {
    display: flex;
    flex-wrap: wrap;
    gap: 20px;
    justify-content: center;
}

.image-card {
    background: #fff;
    padding: 10px;
    border-radius: 8px;
    box-shadow: 0px 2px 10px rgba(0, 0, 0, 0.1);
    text-align: center;
    transition: transform 0.2s;
}

.image-card:hover {
    transform: scale(1.05);
}

.image-card img {
    width: 200px;
    border-radius: 5px;
}

.image-card p {
    font-size: 14px;
    margin-top: 5px;
    color: #333;
}
.data{
    font-weight: bold;
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
                }
                .geolocation{
                    width: 100%;
                    font-size: 10px;
                    text-align: left;
                    margin-top: 5px;
                }
            </style>
        </head>
        <body>
            <h1>Imagens de Sensoriamento</h1>
            <form action="/upload" method="post"
enctype="multipart/form-data">
                <input type="file" name="image" required>
                <input type="text" name="latitude"
placeholder="Digite a Latitude" required>
                <input type="text" name="longitude"
placeholder="Digite a Longitude" required>
                <button type="submit">Enviar</button>
            </form>
            <h2>Imagens enviadas</h2>
            <div class="gallery">
                {% for image in images %}
                <div class="image-card">
                    <p class="data">Data: {{
image[1][:19].replace('T', ' ') }} </p>
                    <img src="{{ url_for('static',
filename='uploads/' + image[0]) }}" alt="Imagem">
                    <p class="geolocation">Latitude: {{ image[2]
}}</p>
                    <p class="geolocation">Longitude: {{ image[3]
}}</p>
                </div>
                {% endfor %}
            </div>
        </body>
        </html>
```

Código-fonte 5 - index.html
Fonte: Elaborado pelo autor (2025)

No código anterior, podemos observar o template html que exibirá imagens juntamente com suas latitudes e longitudes.

O código completo da aplicação encontra-se no GitHub, basta acessar o site <https://github.com/proffernandotfernandes/agriscanner>. O arquivo para construção e disponibilização da imagem (Dockerfile) está disponibilizado a seguir.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
# Usa a imagem oficial do Python como base
FROM python:3.11-slim

# Define o diretório de trabalho dentro do contêiner
WORKDIR /app

# Copia os arquivos da aplicação para o contêiner
COPY . .

# Instala as dependências do Python
RUN pip install --no-cache-dir -r requirements.txt
RUN pip install gunicorn

# Cria a pasta de uploads (caso não exista) e define as permissões
RUN mkdir -p static/uploads && chmod -R 777 static/uploads

# Expõe a porta 5000 para acesso à aplicação
EXPOSE 5000

# Usa Gunicorn para rodar o Flask em produção
CMD ["gunicorn", "-w", "4", "-b", "0.0.0.0:5000", "agriapp:app"]
```

Código-fonte 6 - Código Dockerfile
Fonte: Elaborado pelo autor (2025)

No Dockerfile iremos utilizar uma imagem enxuta que suporte Python. Depois precisamos instalar as dependências do projeto e o servidor web gunicorn. Também será necessário expor a porta 5000 para acesso à aplicação web disponibilizada.

4.4 Criação da pilha do CloudFormation

Agora vamos acessar o serviço CloudFormation na AWS para criar uma pilha que cria uma tarefa para implantação do contêiner. Acesse o CloudFormation e clique no menu “Pilhas” e, em seguida, vá em “Criar pilha”.

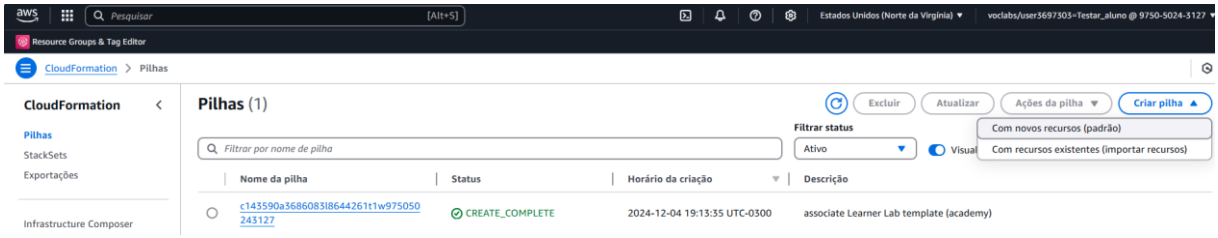

Figura 28 - Criar pilha
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Selecione a opção “Fazer upload de um arquivo de modelo” e escolha o arquivo salvo agriappdemo.yml. Depois, clique em avançar.

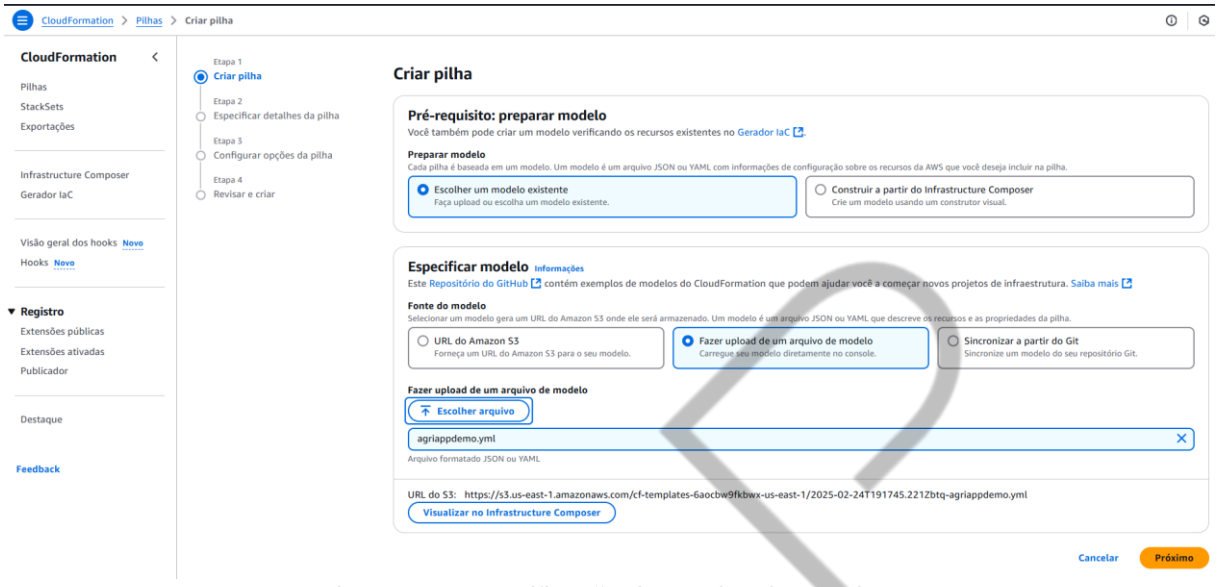

Figura 29 - Especificação de arquivo de template
Fonte: Elaborado pelo autor (2025)

Em seguida, defina um nome para a pilha:

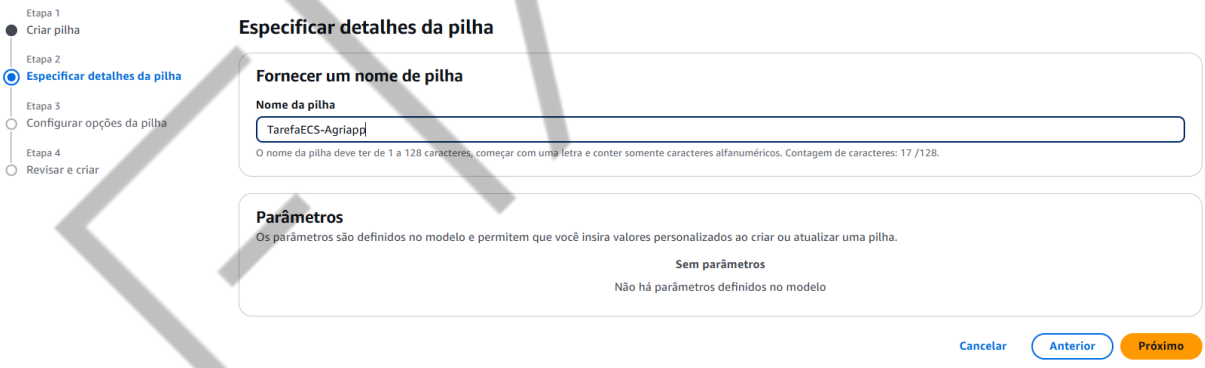

Figura 30 - Nome da pilha no CloudFormation
Fonte: Elaborado pelo autor (2025)

Clique em próximo. Escolha a função do IAM LabRole e clique em “Próximo”:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

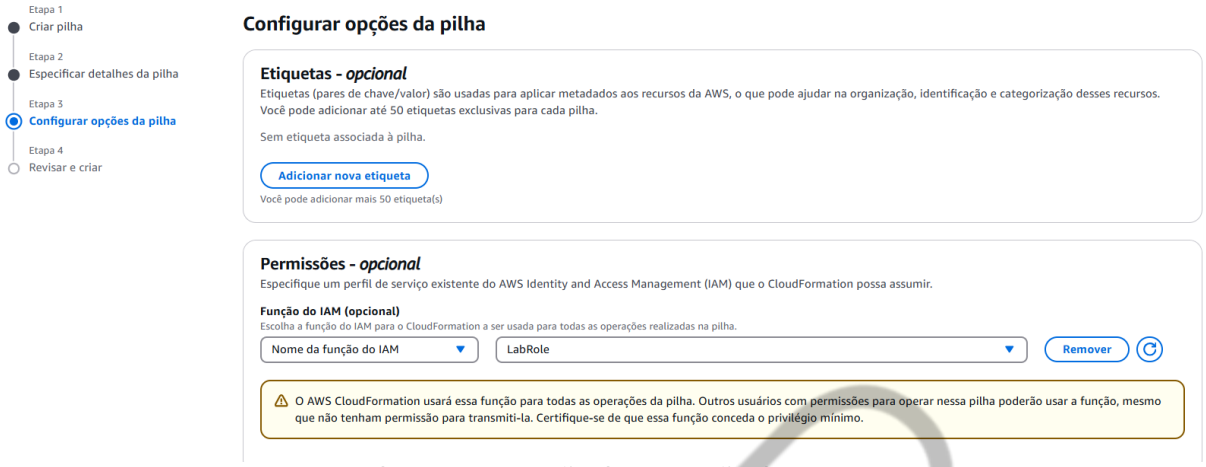

Figura 31 - Função de execução do IAM
Fonte: Elaborado pelo autor (2025)

Confira os detalhes definidos e clique em enviar. Em seguida, acesse o ECS e veja se consta a pilha em definições de tarefa.

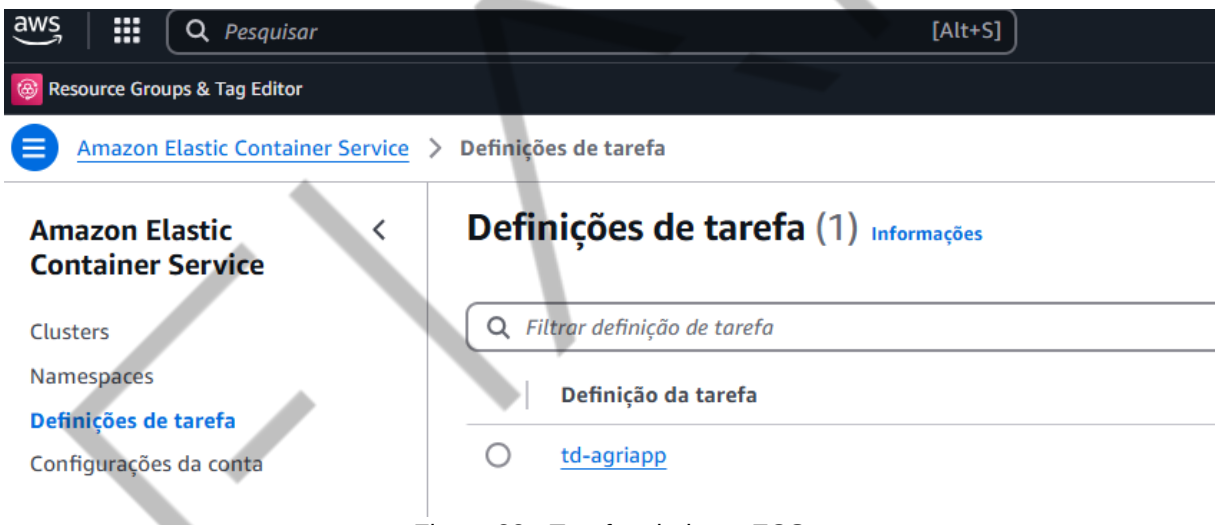

Figura 32 - Tarefa criada no ECS
Fonte: Elaborado pelo autor (2025)

Agora basta executar a tarefa no cluster.

Figura 33 - Execução de tarefa no ECS
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A aplicação utiliza a porta 5000, portanto é necessário liberar a porta 5000 no seu *security group* da VPC padrão, conforme visto na aplicação de demonstração.

Após a liberação da porta 5000, clique no nome da tarefa para visualizar o IP público e acesse a aplicação digitando <http://IPPublico>, substituindo o IPPublico pelo número IP que consta em seu contêiner.

Figura 34 - Aplicação implantada
Fonte: Elaborado pelo autor (2025)

Pronto! Concluímos a segunda etapa da nossa solução, utilizando uma pilha do CloudFormation para subir uma aplicação web completa.

4.5 Exclusão de pilha do CloudFormation

Caso não precise mais acessar os recursos da pilha, você pode removê-la no CloudFormation.

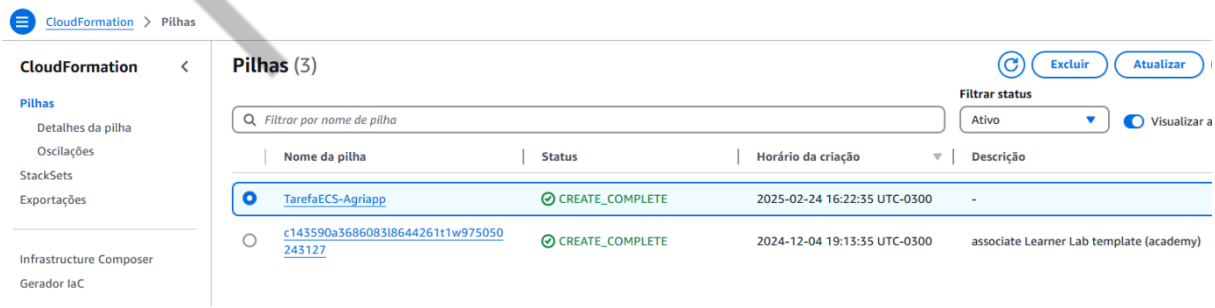

Figura 35 - Exclusão de pilha no CloudFormation
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Aguarde a exclusão da pilha. Em seguida, consulte as definições de tarefa. A tarefa deve ter sido excluída, da mesma forma que o contêiner deve ter sido interrompido.

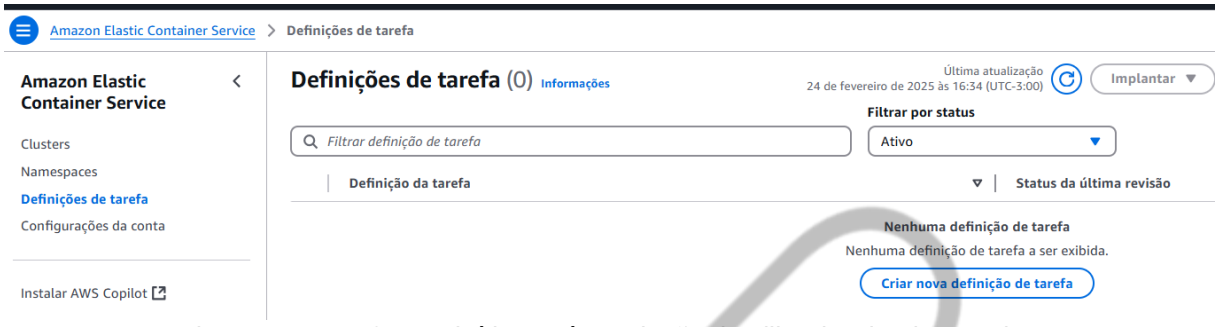

Figura 36 - Tarefas excluídas após exclusão de pilha do CloudFormation
Fonte: Elaborado pelo autor (2025)

Pronto! Agora não existe mais nenhuma tarefa criada.

FIAP

CONCLUSÃO

O uso de contêineres na AWS, especialmente com serviços como ECS (Elastic Container Service) e AWS Fargate, proporciona uma solução altamente escalável, gerenciável e eficiente para implantação de aplicações. Os contêineres eliminam preocupações com configuração manual de servidores e oferecem uma abordagem padronizada para empacotamento, distribuição e execução de aplicações, garantindo portabilidade e otimização de recursos.

Aliado a isso, a Infraestrutura como Código (IAC) com o AWS CloudFormation permite a automação e reprodutibilidade da infraestrutura, reduzindo erros manuais e melhorando a escalabilidade. Com templates YAML ou JSON, é possível provisionar clusters ECS, definir tarefas, criar redes e configurar permissões de forma organizada e versionada. Isso facilita tanto a implantação contínua (CI/CD) quanto a gestão de ambientes complexos, garantindo eficiência operacional e governança.

A combinação de contêineres e CloudFormation capacita equipes a desenvolver, testar e escalar aplicações de maneira rápida e segura, promovendo automação, flexibilidade e controle na nuvem AWS.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

ARQUITETURA Medalhão. **DataBricks**, 2025. Disponível em: <https://www.databricks.com/br/glossary/medallion-architecture>. Acesso em: 14 mar. 2025.

AWS Academy. **AWS Academy**, 2025. Disponível em: <https://awsacademy.instructure.com/>. Acesso em: 14 mar. 2025.

CULKIN, J; ZAZON, M. **AWS Cookbook. Recipes for Success on AWS**. O' Reilly, 2022.

EAGAR, G. **Data Engineering with AWS.** Birmingham, UK: Packt Publishing, 2021.

ELMASRI, R.; NAVATHE, S. B. **Sistemas de Banco de Dados**. 7º Ed.; São Paulo: Pearson Education do Brasil, 2018.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

GLOSSÁRIO

AWS	Amazon Web Services.
CLI	Command Line Interface.
EBS	Elastic Block Storage.
EC2	Elastic Compute Cloud.
EFS	Elastic File System.
RDS	Relational Database Service.
ELT	Extração, Carga e Transformação.
ETL	Extração, Transformação e Carga.
S3	Amazon Simple Storage Service.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com